

FIAP — Ciência da Computação  
Atividade Integradora

# MISSÃO AURORA SIGER

## Relatório Operacional de Pré-Decolagem | Documentação Técnica

*Verificação de telemetria, análise energética e apoio de IA, implementados em  
Python*

### Grupo 18

Felipe de Barros Franco  
Guilherme Santos

São Paulo — 2026

# 1. Introdução

Este documento descreve a atividade integradora "Missão Aurora Siger", que simula o processo de verificação de pré-decolagem de uma nave espacial. O sistema recebe a telemetria da nave (temperaturas, integridade estrutural, energia, pressão dos tanques e status dos módulos críticos), decide entre "PRONTO PARA DECOLAR" e "DECOLAGEM ABORTADA" com base em faixas de segurança predefinidas, calcula a autonomia energética inicial da missão e usa um assistente de IA como camada adicional de leitura sobre os mesmos dados.

O objetivo não é só produzir um veredito automatizado, mas também demonstrar boas práticas de engenharia de software num problema de decisão crítica: separação entre lógica de verificação e apresentação, código modular e testável, controle de versão, documentação rastreável e uso consciente de IA como apoio, e nunca como substituto da decisão técnica.

## 2. Arquitetura e Tecnologias Utilizadas

### 2.1. Linguagem e Execução

- Linguagem: Python 3.9 ou superior.
- Sem dependências externas, apenas biblioteca padrão.
- Entrada de dados via `input()` no terminal, simulando a leitura de sensores.

### 2.2. Estrutura do Projeto

- `main.py`: leitura da telemetria, cálculos de energia e chamada das verificações
- `service.py`: funções de validação de cada parâmetro e orquestração da verificação de pré-decolagem
- `Analise_Pre_Decolagem_Aurora_Siger.ipynb`: relatório escrito com todos os itens da atividade (1.1 a 1.6)
- `imagens/`: fluxograma do algoritmo e gráficos da análise energética

A separação entre `main.py` e `service.py` segue um princípio simples: `service.py` só decide (recebe valores, devolve validade e mensagens de erro) e não imprime nada por conta própria; `main.py` cuida da entrada, da apresentação e da orquestração. Isso torna a verificação testável de forma isolada, sem depender do terminal.

### 2.3. Controle de Versão e Repositório

O projeto é versionado em Git, com o histórico de commits disponível no repositório público do GitHub referenciado na seção 10 deste documento.

### 3. Mapeamento dos Requisitos da Atividade (Checklist de Conformidade)

| Item       | Descrição                                                  | Onde está                                |
|------------|------------------------------------------------------------|------------------------------------------|
| 1.1        | Organização e descrição da telemetria                      | Seção 4 deste documento e notebook       |
| 1.2        | Algoritmo de verificação (fluxograma / pseudocódigo)       | Seção 5 deste documento e notebook       |
| 1.3        | Script em Python com leitura, verificação e resultado      | main.py / service.py e notebook          |
| 1.4        | Análise energética (autonomia inicial)                     | Seção 7 deste documento e notebook       |
| 1.5        | Análise assistida por IA (classificação, anomalias, risco) | Seção 8 deste documento e notebook       |
| 1.6        | Reflexão crítica (ética, impacto social, sustentabilidade) | Seção 9 deste documento e notebook       |
| Entregável | Relatório em PDF                                           | Relatorio_Pre_Decolagem_Aurora_Siger.pdf |
| Entregável | Repositório público no GitHub com notebook e README        | Ver seção 10                             |

### 4. Organização e Descrição da Telemetria (item 1.1)

Durante a janela de pré-decolagem, os sensores da nave enviam um conjunto fixo de leituras. A tabela a seguir lista cada parâmetro monitorado, sua unidade e a faixa considerada segura, as mesmas faixas usadas pelo algoritmo de decisão da seção 5.

| Parâmetro              | Unidade | Faixa segura | Descrição                                                 |
|------------------------|---------|--------------|-----------------------------------------------------------|
| Temperatura interna    | °C      | 18 a 27      | Temperatura da cabine e da eletrônica embarcada           |
| Temperatura externa    | °C      | -60 a 45     | Temperatura do casco, usada para avaliar estresse térmico |
| Integridade estrutural | 0/1     | deve ser 1   | Indica se o casco e as junções estão íntegros             |

| Parâmetro                   | Unidade        | Faixa segura | Descrição                                                    |
|-----------------------------|----------------|--------------|--------------------------------------------------------------|
| Nível de energia            | %              | mínimo 40%   | Carga atual da bateria principal                             |
| Pressão dos tanques         | bar            | 150 a 200    | Pressurização dos propelentes (oxidante e combustível)       |
| Status dos módulos críticos | 0/1 por módulo | todos = 1    | Navegação, comunicação, propulsão, suporte de vida, aviônica |

No script, essas leituras são agrupadas em um dicionário que representa um instante de telemetria. Dois cenários foram construídos para os testes: um nominal, com todos os valores dentro da faixa, e um com falhas plantadas, usado para validar o caminho de aborto.

## 5. Algoritmo de Verificação (item 1.2)

A verificação percorre todos os parâmetros e acumula as não conformidades encontradas numa lista, em vez de parar na primeira falha. Lista vazia ao final significa "PRONTO PARA DECOLAR"; qualquer item na lista significa "DECOLAGEM ABORTADA". Essa escolha custa o mesmo processamento e devolve, de uma só vez, tudo o que precisa ser corrigido antes de uma nova tentativa.

### Pseudocódigo

```

INÍCIO
LER temperatura_interna, temperatura_externa, integridade_estrutural,
    nivel_energia, pressao_tanque, modulos_criticos[]

falhas <- lista vazia

SE temperatura_interna NÃO ESTÁ ENTRE 18 E 27 ENTÃO
    ADICIONAR falha "temperatura interna fora da faixa"
SE temperatura_externa NÃO ESTÁ ENTRE -60 E 45 ENTÃO
    ADICIONAR falha "temperatura externa fora da faixa"
SE integridade_estrutural != 1 ENTÃO
    ADICIONAR falha "integridade estrutural comprometida"
SE nivel_energia < 40 ENTÃO
    ADICIONAR falha "energia insuficiente"
SE pressao_tanque NÃO ESTÁ ENTRE 150 E 200 ENTÃO
    ADICIONAR falha "pressão do tanque fora da faixa"
PARA CADA modulo EM modulos_criticos
    SE modulo != 1 ENTÃO
        ADICIONAR falha "módulo crítico com falha: " + nome_modulo

SE falhas está vazia ENTÃO status <- "PRONTO PARA DECOLAR"
SENÃO status <- "DECOLAGEM ABORTADA"

IMPRIMIR status e falhas (se houver)
FIM

```

# Fluxograma

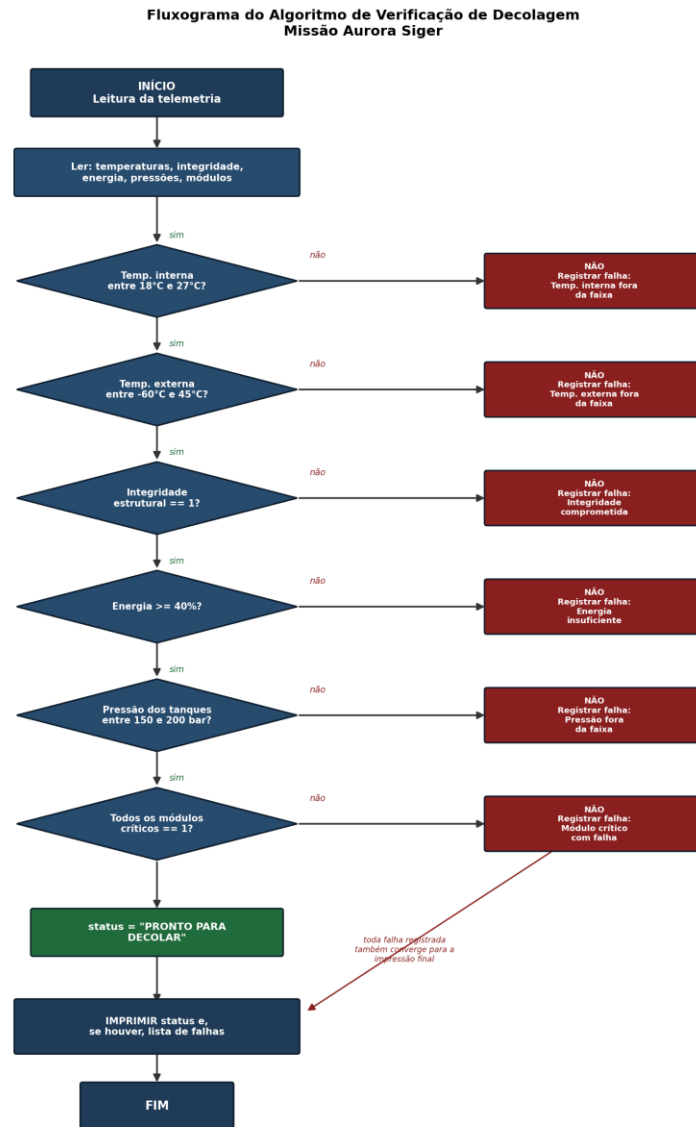


Figura 1 — Fluxograma do algoritmo de verificação de pré-decolagem.

## 6. Script em Python (item 1.3)

O algoritmo foi implementado em `service.py`, com uma função de verificação por parâmetro e uma função orquestradora, `pre_decolagem()`, que reúne os resultados. `main.py` lê a telemetria, chama a orquestradora e imprime o relatório final.

| Função                                               | Responsabilidade                                         |
|------------------------------------------------------|----------------------------------------------------------|
| <code>verificar_temperatura_interna / externa</code> | Valida cada temperatura contra sua faixa segura          |
| <code>verificar_integridade_estrutural</code>        | Confirma se o casco está íntegro (valor 1)               |
| <code>verificar_nivel_energia</code>                 | Confirma energia mínima de 40% para decolagem            |
| <code>verificar_pressao_tanque</code>                | Valida a pressão do tanque entre 150 e 200 bar           |
| <code>verificar_modulos_criticos</code>              | Confirma que os módulos críticos estão operacionais      |
| <code>pre_decolagem(...)</code>                      | Executa todas as verificações e devolve a lista de erros |
| <code>decolagem(erros)</code>                        | Imprime o veredito final a partir da lista de erros      |

### 6.1. Apresentação de Evidências

O projeto é versionado em Git, com o histórico de commits disponível no repositório público do GitHub referenciado na seção 10 deste documento.

## Evidência de execução: cenário nominal

```
python3 main.py

Digite a temperatura interna (deve estar entre 18.0°C e 27.0°C): 21.5
Digite a temperatura externa (deve estar entre -60.0°C e 45°C): -42.3
Digite 1 caso integridade Ok: 1
Digite a pressão dentro do tanque (deve estar entre 150 bar e 200 bar): 182
Digite 1 caso módulos críticos Ok: 1
Digite o percentual de carga atual (deve ser maior que 40%): 87
Digite a porcentagem de perdas energéticas estimadas: 6

ORGANIZAÇÃO E DESCRIÇÃO DA TELEMETRIA
-----
Temperatura interna.....: 21.5 °C
Temperatura externa.....: -42.3 °C
Integridade estrutural (0/1).....: 1
Pressão dos tanques.....: 182.0 bar
Status dos módulos críticos (0/1).....: 1
-----

ANÁLISE ENERGÉTICA
-----
Capacidade total da bateria.....: 150.0 kWh
Carga atual (telemetria).....: 87%
Energia disponível.....: 130.50 kWh
Perdas energéticas (6%).....: 7.83 kWh
Energia útil (após perdas).....: 122.67 kWh
Consumo estimado na decolagem.....: 30.00 kWh
Energia restante pós-decolagem.....: 92.67 kWh
-----

Verificação de temperatura interna iniciada.
Temperatura interna válida!
Verificação de temperatura externa iniciada.
Temperatura externa válida!
Verificação de integridade estrutural iniciada.
Integridade estrutural válida!
Verificação de nível de energia iniciada.
Nível de energia válido!
Verificação de pressão do tanque iniciada.
Pressão do tanque válida!
Verificação dos módulos críticos iniciada.
Módulos críticos válidos!
-----

DECOLAGEM AUTORIZADA
-----
```

Figura 2 — Saída do script com telemetria dentro das faixas seguras.

## Evidência de execução: cenário com falhas

```
python3 main.py

Digite a temperatura interna (deve estar entre 18.0°C e 27.0°C): 29.8
Digite a temperatura externa (deve estar entre -60.0°C e 45°C): -41.0
Digite 1 caso integridade 0k: 1
Digite a pressão dentro do tanque (deve estar entre 150 bar e 200 bar): 141
Digite 1 caso módulos críticos 0k: 0
Digite o percentual de carga atual (deve ser maior que 40%): 33
Digite a porcentagem de perdas energéticas estimadas: 6

ORGANIZAÇÃO E DESCRIÇÃO DA TELEMETRIA
-----
Temperatura interna.....: 29.8 °C
Temperatura externa.....: -41.0 °C
Integridade estrutural (0/1).....: 1
Pressão dos tanques.....: 141.0 bar
Status dos módulos críticos (0/1).....: 0

ANÁLISE ENERGÉTICA
-----
Capacidade total da bateria.....: 150.0 kWh
Carga atual (telemetria).....: 33%
Energia disponível.....: 49.50 kWh
Perdas energéticas (6%).....: 2.97 kWh
Energia útil (após perdas).....: 46.53 kWh
Consumo estimado na decolagem.....: 30.00 kWh
Energia restante pós-decolagem.....: 16.53 kWh

Verificação de temperatura interna iniciada.
Temperatura interna inválida!
Verificação de temperatura externa iniciada.
Temperatura externa válida!
Verificação de integridade estrutural iniciada.
Integridade estrutural válida!
Verificação de nível de energia iniciada.
Nível de energia insuficiente!
Verificação de pressão do tanque iniciada.
Pressão do tanque inválida!
Verificação dos módulos críticos iniciada.
Um ou mais módulos críticos com falha!

DECOLAGEM NÃO AUTORIZADA, VEJA OS ERROS!
-----
Temperatura interna fora da faixa segura: 29.8°C (esperado 18.0-27.0)
Nível de energia insuficiente: 33.0% (mínimo exigido: 40%)
Pressão do tanque fora da faixa segura: 141.0 bar (esperado 150-200)
Um ou mais módulos críticos com falha
```

Figura 3 — Saída do script com quatro não conformidades plantadas na telemetria.

## 7. Análise Energética (item 1.4)

A autonomia inicial da nave é calculada em quatro passos: energia disponível a partir da carga percentual, energia útil após descontar perdas, energia restante após o consumo da decolagem, e autonomia em horas dividindo o restante pelo consumo médio de cruzeiro.

- Capacidade total (kWh): energia máxima armazenada pela bateria principal
- Carga atual (%): valor lido da telemetria no momento da checagem
- Consumo estimado na decolagem (kWh): energia gasta pelos sistemas na janela de decolagem
- Perdas energéticas (%): dissipação térmica e ineficiência de conversão/inversores



| Variável                       | Valor       |
|--------------------------------|-------------|
| Capacidade total da bateria    | 150,0 kWh   |
| Carga atual (telemetria)       | 87%         |
| Energia disponível             | 130,50 kWh  |
| Perdas energéticas (6%)        | 7,83 kWh    |
| Energia útil (após perdas)     | 122,67 kWh  |
| Consumo estimado na decolagem  | 42,00 kWh   |
| Energia restante pós-decolagem | 80,67 kWh   |
| Autonomia estimada             | 25,21 horas |

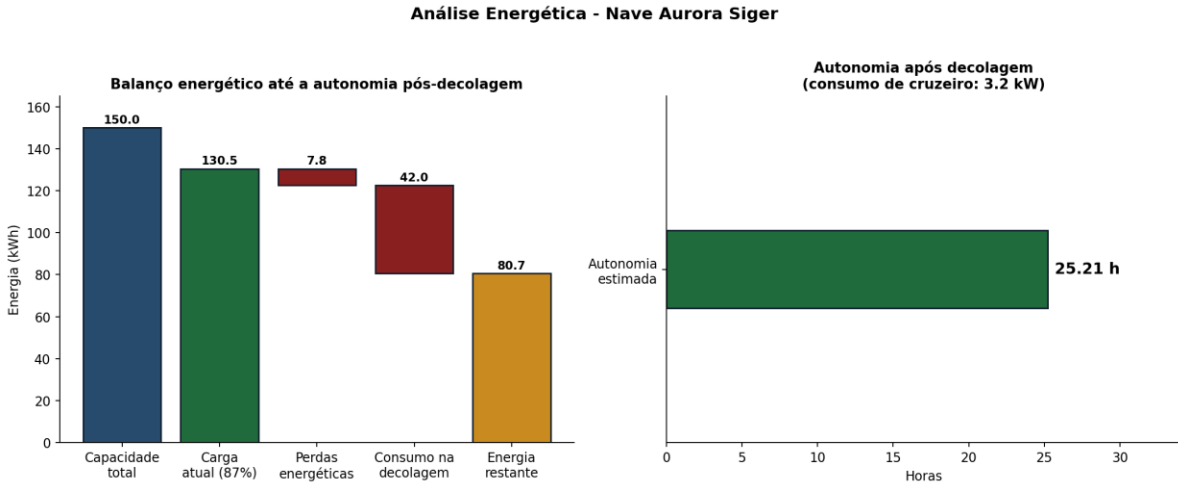


Figura 4 — Composição da energia disponível e autonomia estimada da missão.

Com 87% de carga numa bateria de 150 kWh, a nave parte com 130,5 kWh; descontadas as perdas de 6% e os 42 kWh da decolagem, restam 80,67 kWh, suficientes para cerca de 25,21 horas de operação em regime de cruzeiro. A carga de 87% é folgada frente ao mínimo de 40% exigido para autorizar a decolagem, mas esse mínimo garante apenas viabilidade, não conforto de autonomia depois dela.

## 8. Análise Assistida por IA (item 1.5)

Além da verificação determinística por faixas, a telemetria dos dois cenários foi submetida a um assistente de IA, com o pedido de classificar cada parâmetro (NOMINAL, ATENÇÃO ou CRÍTICO), identificar anomalias entre parâmetros, não apenas violações isoladas, e sugerir um nível de risco geral com recomendação.

### Cenário 1 — telemetria nominal

| Parâmetro           | Valor             | Classificação                |
|---------------------|-------------------|------------------------------|
| Temperatura interna | 21,5 °C           | NOMINAL                      |
| Nível de energia    | 87%               | NOMINAL (margem confortável) |
| Pressão dos tanques | 182,0 / 178,5 bar | NOMINAL                      |
| Módulos críticos    | todos = 1         | NOMINAL                      |

Anomalias identificadas: nenhuma. Sugestão de risco: BAIXO, recomendação de prosseguir com a contagem regressiva.

### Cenário 2 — telemetria com falhas

| Parâmetro                  | Valor     | Classificação                     |
|----------------------------|-----------|-----------------------------------|
| Temperatura interna        | 29,8 °C   | CRÍTICO (2,8 °C acima do limite)  |
| Nível de energia           | 33%       | CRÍTICO (7 p.p. abaixo do mínimo) |
| Pressão tanque combustível | 141,0 bar | CRÍTICO (9 bar abaixo do mínimo)  |
| Módulo "propulsão"         | 0         | CRÍTICO (módulo inoperante)       |

Além das violações isoladas, a IA cruzou parâmetros e apontou duas anomalias adicionais: a combinação de temperatura alta com energia baixa (indício de falha no sistema de refrigeração) e a diferença de 41 bar entre os dois tanques de propelente, muito acima da variação de referência vista no cenário nominal (indício de vazamento ou falha de pressurização). Sugestão de risco: ALTO, recomendação de abortar a decolagem e inspecionar propulsão e tanque de combustível antes de nova tentativa.

A IA atua aqui como camada de apoio à decisão, nunca como substituta do algoritmo das seções 5 e 6: o algoritmo é determinístico e sempre responde da mesma forma para a mesma entrada, requisito básico de um sistema de segurança. A IA acrescenta uma leitura qualitativa, cruzando parâmetros que as regras de faixa avaliam isoladamente.

Vale registrar que a IA também apoiou etapas do próprio desenvolvimento deste projeto, discussão de desenho antes do código, revisão da separação entre validação e apresentação, e geração da documentação a partir de decisões já tomadas e testadas. Em nenhuma etapa a IA decidiu por conta própria: a responsabilidade técnica final permaneceu aos integrantes, quem escreveu e testou o código. Mais detalhes sobre esse processo estão no README.md do repositório.

## **9. Reflexão Crítica (item 1.6)**

### **Ética e responsabilidade**

Automatizar a decisão entre autorizar e abortar uma decolagem transfere para o código uma responsabilidade que, numa missão real, envolve vidas e recursos de alto valor. As faixas seguras precisam ter origem em critério de engenharia, não em números escolhidos por conveniência, e o sistema precisa ser auditável, por isso cada não conformidade é registrada individualmente, em vez de um simples sim/não sem justificativa. A IA usada na seção 8 exige o mesmo cuidado: modelos de linguagem produzem, às vezes, conclusões com aparência técnica, mas sem fundamento; suas sugestões orientam a decisão humana, mas não a substituem.

### **Impacto social da exploração espacial**

Missões como a Aurora Siger dependem de investimento alto em ciência, engenharia e formação especializada, recurso que poderia, alternativamente, ser aplicado em problemas terrestres imediatos. Por outro lado, tecnologias nascidas da exploração espacial costumam retornar à sociedade em usos civis, como sensores miniaturizados, materiais mais leves e algoritmos de decisão autônoma. A questão central não é escolher entre uma coisa e outra, mas garantir que o retorno dessa tecnologia não se concentre apenas em poucas empresas ou países.

### **Sustentabilidade tecnológica**

A análise energética da seção 7 ilustra, em escala pequena, uma restrição válida para qualquer sistema computacional: a energia disponível é finita e conhecida. Numa nave isso é evidente, mas o mesmo raciocínio vale para um data center na Terra. Escrever algoritmos eficientes é, portanto, mais que boa prática de engenharia: cada unidade de energia economizada em processamento é energia que não precisou ser gerada, transmitida e armazenada. Isso vale para o projeto físico de uma missão espacial, em que geração renovável embarcada e controle de resíduos funcionam melhor como requisito desde o início do que como ajuste posterior.

## 10. Instruções de Execução

O script roda direto, sem dependências externas:

```
git clone https://github.com/felipxv/FIAP-AuroraSiger_PreDecolagem.git
cd FIAP-AuroraSiger_PreDecolagem
python3 main.py
```

O programa solicita os valores de telemetria e de carga/perdas energéticas via terminal, executa as verificações e imprime a tabela de telemetria, a análise energética e o veredito final ("DECOLAGEM AUTORIZADA" ou lista de não conformidades). O notebook Analise\_Pre\_Decolagem\_Aurora\_Siger.ipynb reúne o relatório escrito dos itens 1.1 a 1.6 e pode ser aberto no Jupyter, VS Code ou Google Colab, ele contém apenas texto e imagens, não é necessário executá-lo.

**Repositório:** [https://github.com/felipxv/FIAP-AuroraSiger\\_PreDecolagem.git](https://github.com/felipxv/FIAP-AuroraSiger_PreDecolagem.git)

## 11. Decisões de Projeto e Desafios

### 11.1. Verificar tudo antes de decidir

**Desafio:** o algoritmo poderia parar no primeiro parâmetro fora da faixa, economizando processamento.

**Decisão:** manter a verificação completa mesmo após encontrar uma falha, para que a equipe de missão receba, de uma vez, tudo o que precisa corrigir antes de uma nova tentativa, visível no cenário de teste com falhas, que reporta as quatro não conformidades juntas.

### 11.2. Separar decisão de apresentação

**Desafio:** manter a lógica de verificação testável e reutilizável, sem depender do terminal.

**Decisão:** `pre_decolagem()` apenas recebe valores e devolve uma lista de erros, sem imprimir nada; `decolagem()` cuida só da apresentação. Isso permite reaproveitar a verificação em outro contexto (por exemplo, um teste automatizado) sem alterar a lógica de negócio.

### 11.3. Faixas seguras como parâmetro, não como número solto

**Desafio:** evitar que valores de engenharia fiquem espalhados e difíceis de auditar dentro da lógica.

**Decisão:** cada verificação documenta, no próprio código e no relatório, a faixa segura usada e sua origem, para que a decisão automatizada permaneça rastreável.

## 12. Considerações Finais

O desenvolvimento da Missão Aurora Siger permitiu aplicar, de forma integrada, pensamento computacional, boas práticas de estruturação de código em Python, cálculo aplicado de autonomia energética e uso responsável de IA como camada de apoio à decisão. A separação entre verificação determinística e leitura qualitativa por IA reforça um princípio importante para qualquer sistema que automatiza decisões de risco: a IA amplia a análise, mas não substitui a responsabilidade técnica de quem constrói e valida o sistema. Esse mesmo princípio guiou como a IA foi usada durante a construção deste projeto, como parceira de raciocínio em cada etapa, do desenho à documentação, sem transferir a ela a decisão final.

## 13. Referências

Python Software Foundation. Python 3 Documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: set. 2026.

FIAP. Atividade Integradora — Pensamento Computacional, IA & Prompt Engineering, Energias Renováveis e Sustentáveis, Formação Social e Sustentabilidade. Material didático. São Paulo: FIAP, 2026.

Git. Git Documentation. Disponível em: <https://git-scm.com/doc>. Acesso em: set. 2026.

GitHub. GitHub Docs. Disponível em: <https://docs.github.com>. Acesso em: set. 2026.

Jupyter Project. Jupyter Notebook Documentation. Disponível em: <https://docs.jupyter.org>. Acesso em: set. 2026.

NASA. National Aeronautics and Space Administration — página oficial. Disponível em: <https://www.nasa.gov>. Acesso em: set. 2026.

NASA. Humans in Space (segurança e critérios de decisão em voos tripulados, referência conceitual para o veredito GO/NO-GO adotado neste projeto). Disponível em: <https://www.nasa.gov/humans-in-space>. Acesso em: set. 2026.

SpaceX. Página oficial — foguetes, missões e transmissões de lançamento. Disponível em: <https://www.spacex.com>. Acesso em: set. 2026.

ESA — European Space Agency. Página oficial. Disponível em: <https://www.esa.int>. Acesso em: set. 2026.

ESA. Space Debris (Space Safety) — sustentabilidade e uso responsável do ambiente orbital, referência conceitual para a reflexão sobre sustentabilidade tecnológica. Disponível em: [https://www.esa.int/Space\\_Safety/Space\\_Debris](https://www.esa.int/Space_Safety/Space_Debris). Acesso em: set. 2026.